

- AWS Glue PySpark extensions

- ▼ I/O in awsglue

```
from awsglue.context import GlueContext
from awsglue.transforms import ApplyMapping
from awsglue.dynamicframe import DynamicFrame
from pyspark.context import SparkContext
import sys
```

```
# Initialize Spark and Glue contexts
sc = SparkContext.getOrCreate()
glueContext = GlueContext(sc)
```

READ

- i. **create_dynamic_frame.from_catalog()** : Load a table from the Glue Data Catalog.

```
# Read data from Glue Data Catalog
dynamic_frame_catalog = glueContext.create_dynamic_frame.from_catalog(
    database="my_database",
    table_name="my_table"
)
dynamic_frame_catalog.show()
```

- ii. **create_dynamic_frame.from_options()** : Load data from other sources, such as S3, JDBC, or other databases.

```
# Read JSON data from S3
dynamic_frame_s3 = glueContext.create_dynamic_frame.from_options(
    connection_type="s3",
    connection_options={"paths": ["s3://my-bucket/my-folder/" ]},
    format="json"
)
dynamic_frame_s3.show()
```

- iii. **create_dynamic_frame.from_options()** : To read data from a JDBC source (like PostgreSQL, MySQL, etc.) into a DynamicFrame, use the `create_dynamic_frame.from_options` method.

```

# Set up JDBC connection options
jdbc_options = {
    "url": "jdbc:mysql://hostname:port/database", # JDBC URL for the database
    "user": "username", # Database username
    "password": "password", # Database password
    "dbtable": "table_name", # Table to read from
    "driver": "com.mysql.cj.jdbc.Driver" # JDBC driver class name
}

# Read data from JDBC into a DynamicFrame
dynamic_frame = glueContext.create_dynamic_frame.from_options(
    connection_type="jdbc",
    connection_options=jdbc_options
)

```

iv. **create_dynamic_frame.from_rdd()** : Create a DynamicFrame from an RDD.

```

# Create an RDD from a list
rdd = sc.parallelize([
    {"name": "Alice", "age": 30},
    {"name": "Bob", "age": 45}
])

# Convert RDD to DynamicFrame
dynamic_frame_rdd = glueContext.create_dynamic_frame.from_rdd(rdd, "rdd_dynamic")
dynamic_frame_rdd.show()

```

v. **create_dynamic_frame.from_dataframe()** : Convert a Spark DataFrame to a DynamicFrame .

```

from pyspark.sql import SparkSession
from pyspark.sql import Row

# Create a Spark DataFrame
spark = SparkSession.builder.getOrCreate()
df = spark.createDataFrame([Row(name="Charlie", age=50), Row(name="Diana", age=40)])

# Convert Spark DataFrame to Glue DynamicFrame
dynamic_frame_df = glueContext.create_dynamic_frame.from_dataframe(df, glueContext)
dynamic_frame_df.show()

```

WRITE

- i. **write_dynamic_frame.from_options()** : Write a `DynamicFrame` to a destination, such as S3.

```
# Write DynamicFrame to S3 as JSON
glueContext.write_dynamic_frame.from_options(
    frame=dynamic_frame_s3,
    connection_type="s3",
    connection_options={"path": "s3://my-output-bucket/output-folder/"},
    format="json"
)

# Write DynamicFrame to JDBC
glueContext.write_dynamic_frame.from_options(
    frame=dynamic_frame,
    connection_type="jdbc",
    connection_options=jdbc_options_write
)
```

- ii. **write_dynamic_frame.from_catalog()** : Write a `DynamicFrame` to a Glue Data Catalog table.

```
# Write DynamicFrame to Glue Data Catalog
glueContext.write_dynamic_frame.from_catalog(
    frame=dynamic_frame_catalog,
    database="my_output_database",
    table_name="my_output_table"
)
```

- iii. **write_from_options()** : Writes and returns a `DynamicFrame` or `DynamicFrameCollection` that is created with the specified connection and format information.
- iv. **write_data_frame_from_catalog()** : Writes and returns a `DataFrame` using information from a Data Catalog database and table. This method supports writing to data lake formats (Hudi, Iceberg, and Delta Lake)
- v. **get_sink()** : Define a sink (target) for a data output, particularly useful when output specifications require more configuration.

```
# Configure S3 as the output sink
sink = glueContext.getSink(
    connection_type="s3",
    path="s3://my-output-bucket/output-folder/",
    format="parquet"
)

# Write data to the sink
sink.writeFrame(dynamic_frame_catalog)
```

- ▼ **Job API in awsglue**

The `awsglue.job.Job` class in Python is part of the AWS Glue library and is typically used to handle and control ETL jobs within an AWS Glue environment. It provides a way to initiate, manage, and monitor AWS Glue jobs, which are commonly used for transforming and moving data across different data stores on AWS.

- Job Management:** `awsglue.job.Job` allows for starting and stopping Glue jobs, and monitoring their states (e.g., `RUNNING`, `SUCCEEDED`, `FAILED`).
- Parameter Handling:** Glue jobs often require specific parameters to run, and `awsglue.job.Job` makes it easy to fetch these parameters for ETL tasks.
- Integration with AWS Glue ETL Scripts:** The class is designed to integrate well with the Glue job execution framework, making it a convenient option for organizing ETL logic directly in Glue.
- Example:**

```

# Import necessary AWS Glue libraries and Spark
from awsglue.context import GlueContext
from awsglue.job import Job
from awsglue.utils import getResolvedOptions
from awsglue.dynamicframe import DynamicFrame
from pyspark.sql import SparkSession
import sys

# Initialize a Spark session and Glue context
spark = SparkSession.builder.appName("Glue_ETL_Job").getOrCreate()
glueContext = GlueContext(spark.sparkContext)

# Define the expected parameters
args = getResolvedOptions(sys.argv, ['JOB_NAME', 'SOURCE_BUCKET', 'DESTINATION_I

# Initialize the Glue job
job = Job(glueContext)
job.init(args['JOB_NAME'], args)

# Extract parameters
source_bucket = args['SOURCE_BUCKET']          # e.g., 's3://my-source-bucket/data'
destination_bucket = args['DESTINATION_BUCKET'] # e.g., 's3://my-destination-bucket'
partition_date = args['PARTITION_DATE']         # e.g., '2024-11-01'

# Step 1: Extract – Load data as DynamicFrame from the source S3 bucket
source_path = f"{source_bucket}{partition_date}/"
data_dynamic_frame = glueContext.create_dynamic_frame.from_options(
    connection_type="s3",
    connection_options={"paths": [source_path]},
    format="json"
)

# Print schema for debugging purposes
print("Source Data Schema:")
data_dynamic_frame.printSchema()

# Step 2: Transform – Apply transformations on the DynamicFrame
# Selecting specific fields and filtering records
transformed_dynamic_frame = data_dynamic_frame.select_fields(["id", "name", "date"])
transformed_dynamic_frame.filter(lambda row: row["status"] == "active")

# Step 3: Load – Write the transformed data to the destination bucket in Parquet
destination_path = f"{destination_bucket}{partition_date}/"

```

```

glueContext.write_dynamic_frame.from_options(
    frame=transformed_dynamic_frame,
    connection_type="s3",
    connection_options={"path": destination_path},
    format="parquet",
    format_options={"compression": "snappy"}
)

# Commit the Glue job to signal completion
job.commit()

print(f"ETL Job completed. Transformed data is saved to {destination_path}")

```

In the AWS Glue script above, initializing and committing the `Job` object is essential for defining and properly executing an AWS Glue job. Here's why each part—initializing and committing—is important:

i. Initializing the Job Object

```
job = Job(glueContext)
```

- The `Job` object represents the entire Glue job in the context of AWS Glue's infrastructure. Initializing this object with `glueContext` registers it within the AWS Glue service and makes Glue aware that this script is intended to be run as a managed job.
- This initialization step is required if you want the job to be trackable in the AWS Glue console and to take advantage of features such as tracking job metadata, monitoring job runs, and retrying jobs upon failure.

ii. Committing the Job Object

```
job.commit()
```

- Calling `job.commit()` marks the completion of the Glue job, finalizing the job's execution.
- `job.commit()` also performs any last-minute checkpointing necessary for job tracking and logging in AWS Glue. Without it, the job run might be considered incomplete, and AWS Glue won't finalize the execution, which can impact job state reporting and prevent subsequent Glue workflows from proceeding.
- In some cases, `job.commit()` also commits the transaction if Glue is working with data sources that support transactional semantics, ensuring all operations performed during the job are consistent.

- The `job.commit()` call is critical for accurate status reporting and clean resource management within AWS Glue.

iii. **Impact of Omitting `job.commit()`** : If `job.commit()` is omitted, AWS Glue does not recognize the job as fully complete, and this has several possible consequences:

- The job might appear to be stuck in a "running" state in the Glue console, as Glue does not receive the final notification of completion.
- AWS Glue workflows that depend on this job may not proceed, as the job status is not marked as "Completed."
- Any resources provisioned for this job might not be released properly, which could lead to cost inefficiencies.

iv. **Notes:**

- When you create an AWS Glue job in the console or via the CLI, you specify the `JOB_NAME` and any other parameters you need, which are passed to `getResolvedOptions`. These parameters allow the Glue job to access different configurations and data sources dynamically.
- Suppose you need to read data from an S3 bucket, apply a transformation, and save the output to another bucket. You can set up this ETL workflow in Glue with the `awsglue.job.Job` class to manage the job flow, retry on failures, and monitor the job's execution.

v. **Start the Job using AWS SDK:**

```

import boto3

# Initialize the Glue client
glue_client = boto3.client('glue')

# Define the job name and parameters
job_name = "my_etl_job"
job_parameters = {
    "--JOB_NAME": job_name,
    "--SOURCE_BUCKET": "s3://my-source-bucket/data/",
    "--DESTINATION_BUCKET": "s3://my-destination-bucket/processed/",
    "--PARTITION_DATE": "2024-11-01"
}

# Start the Glue job with parameters
response = glue_client.start_job_run(
    JobName=job_name,
    Arguments=job_parameters
)

# Get the Job Run ID to track the job status
job_run_id = response['JobRunId']
print(f"Started Glue job '{job_name}' with JobRunId: {job_run_id}")

```

• ▼ GlueContext API in awsglue

The `GlueContext` is the entry point for AWS Glue functionalities. It wraps around the `SparkContext` and provides access to AWS Glue's features.

- `create_dynamic_frame.from_catalog()` : Reads data from the AWS Glue Data Catalog.
- `create_dynamic_frame.from_options()` : Reads data from various data sources like S3, JDBC, and DynamoDB.
- `create_dynamic_frame.from_rdd()` : Creates a `DynamicFrame` from an RDD.
- `create_dynamic_frame.from_dataframe()` : Converts a Spark `DataFrame` to a Glue `DynamicFrame` .
- `create_data_frame.from_catalog()` : Creates a Spark `DataFrame` from the Glue Data Catalog.
- `create_data_frame.from_options()` : Creates a Spark `DataFrame` from a specified data source.

- `write_dynamic_frame.from_options()` : Writes a `DynamicFrame` to a specific output.
- `write_dynamic_frame.from_catalog()` : Writes a `DynamicFrame` to the Glue Data Catalog.
- `get_sink()` : Gets a sink (destination) for a given `DynamicFrame` .

i. **Job Bookmarks:** Track job bookmarks, which record the last processed state of data.
Useful for incremental loads.

- **Get Bookmark:** `get_bookmark()` retrieves the bookmark for a specific data source.

```
bookmark = glueContext.get_bookmark("my_job_name")
print(bookmark)
```

- **Set Bookmark:** `set_bookmark()` sets a bookmark for tracking processed data.

```
glueContext.set_bookmark("my_job_name", bookmark_state={"last_processed_id": 12}
```

- **Remove Bookmark:** `remove_bookmark()` clears the bookmark.

```
glueContext.remove_bookmark("my_job_name")
```

ii. **GlueContext Extensions:** `create_catalog_table()` and `update_catalog_table()`

interact with the Glue Data Catalog to create or update table definitions.

- **Create a New Table in Glue Catalog:** `create_catalog_table()` creates a new table in the Glue Data Catalog.

```
glueContext.create_catalog_table(
    database="my_database",
    table_name="new_table",
    schema=dynamic_frame_catalog.schema()
)
```

- **Update an Existing Table in Glue Catalog:** `update_catalog_table()` updates an existing table in the Glue Data Catalog.

```
glueContext.update_catalog_table(
    database="my_database",
    table_name="existing_table",
    schema=dynamic_frame_catalog.schema()
)
```

• ▼ **DynamicFrame API in awsglue**

DynamicFrame is a distributed data structure in AWS Glue, which supports schema inference and transformations.

A `DynamicFrame` is a distributed data structure used in AWS Glue, similar to a Spark `DataFrame` but optimized for AWS Glue ETL transformations.

- `fromDF()` : Converts a Spark `DataFrame` to a `DynamicFrame` .
- `drop_fields()` : Drops specific fields from a `DynamicFrame` .
- `map()` : Applies a function to each record.
- `relationalize()` : Flattens nested structures in a `DynamicFrame` .
- `toDF()` : Converts a `DynamicFrame` to a Spark `DataFrame`.

```
df = dynamic_frame.toDF()
```

- `apply_mapping()` : Applies transformations to the columns in a `DynamicFrame` . Maps fields and performs type conversions.

```
mapped_dynamic_frame = dynamic_frame.apply_mapping(  
    [("old_col1", "string", "new_col1", "int"),  
    ("old_col2", "double", "new_col2", "double")]  
)
```

- `resolveChoice()` : Resolves data types ambiguities in a `DynamicFrame` .

```
resolved_dynamic_frame = dynamic_frame.resolveChoice(  
    specs=[("column_name", "cast:int")]  
)
```

- `select_fields()` Selects specific fields from a `DynamicFrame` .

```
selected_dynamic_frame = dynamic_frame.select_fields(["field1", "field2"])
```

- `filter()` : Filters rows based on a condition.

```
filtered_dynamic_frame = dynamic_frame.filter(  
    lambda row: row["column_name"] > 100  
)
```

- `rename_field()` : Renames a field in a `DynamicFrame` .

```
```python
renamed_dynamic_frame = dynamic_frame.rename_field("old_name", "new_
```
```

• ▼ GlueTransform API in awsglue

GlueTransform GlueTransform is a base class for AWS Glue transformations.

- **SelectFields** : Select specific fields from a DynamicFrame .
- **DropFields** : Drop fields from a DynamicFrame .
- **Join** : Join two DynamicFrames .
- **Map** : Apply a custom mapping function to transform data.
- **Filter** : Filter out records based on a specified condition.
- **ApplyMapping** : Used for column renaming and type casting.
- **Join** Joins two DynamicFrames .

```
joined_frame = DynamicFrame.fromDF(
    df1.join(df2, df1["key"] == df2["key"], "inner"), glueContext, "joined_frame"
)
```

- **ApplyMapping** Used for remapping fields and changing data types.

```
transformed_dynamic_frame = ApplyMapping.apply(
    frame=dynamic_frame,
    mappings=[("source_column", "string", "target_column", "int")]
)
```

• ▼ Util API in awsglue

The `awsglue.utils` module provides helper functions for managing job parameters and bookmarks.

- `getResolvedOptions(sys.argv, ['ARG_NAME'])` : Parses job arguments.
- `convert_to_dict()` : Converts a DynamicFrame to a Python dictionary.
- `get_job_bookmark_state()` : Gets the current state of a job bookmark.
- `reset_job_bookmark()` : Resets the job bookmark.
- **getResolvedOptions()** Parses job arguments for flexibility in configuring jobs.

```
from awsglue.utils import getResolvedOptions
import sys
```

```
args = getResolvedOptions(sys.argv, ['JOB_NAME', 'INPUT_PATH'])
```

- **convert_to_dict()** Converts a DynamicFrame to a Python dictionary for easier manipulation.

```
dict_data = dynamic_frame.toDF().collect()
```

- **get_job_bookmark_state()** Retrieves the state of a job bookmark, useful in incremental processing.

```
state = glueContext.get_job_bookmark_state("job_name")
```

• ▼ Glue Data API in awsglue

Glue Data Types: Glue provides several specialized data types to manage data schema in DynamicFrames .

- **ArrayType** Represents array types in schema definition.

```
from awsglue import DynamicFrame
from pyspark.sql.types import ArrayType, StringType
```

```
schema = ArrayType(StringType())
```

- **StructType** Used to define complex nested structures.

```
from pyspark.sql.types import StructType, StructField, StringType
```

```
schema = StructType([StructField("field1", StringType(), True)])
```

• ▼ Notes on awsglue

AWS Glue Job Bookmarks are a powerful incremental data processing feature that tracks the state of previously processed data, so jobs can **pick up where they left off**—instead of

reprocessing everything. Below is a **comprehensive breakdown** of the concepts, components, and best practices.

 What Are AWS Glue Job Bookmarks?

- **Glue Job Bookmarks** enable **stateful ETL jobs** by **persisting metadata** about previous runs (e.g., file paths, timestamps, partitions).
- Bookmarks help avoid **duplicate processing** when reading from:
- Amazon S3
- Amazon DynamoDB
- JDBC sources
- Kafka/Kinesis (in streaming jobs)

 Core Concepts

| Concept | Description |
|-------------------|--|
| Stateful Job | A job that remembers what it previously processed (via bookmarks) |
| Bookmark Metadata | Stores file paths, partition values, and timestamps |
| Checkpointing | For streaming jobs, bookmarks act like checkpoints |
| Run ID | Unique identifier per job run—used to distinguish different ETL cycles |
| Job Name | Bookmarks are stored per job name and per source |

 How It Works

- i. When a Glue job runs with **bookmarks enabled**, it keeps track of:
 - Files/partitions processed
 - Last update timestamps
 - Custom filters (if applied)
- i. In the **next job run**, Glue:
 - **Filters out** already-processed data
 - **Loads only new or changed data**

This is useful for **incremental ETL pipelines** such as daily S3 ingestion jobs.

Enable or Disable Bookmarks

Python Script (Glue Job)

```
job = Job(glueContext)
job.init(args['JOB_NAME'], args)

# Enable Job Bookmark
# glueContext.read.options(...).enableBookmarking()
```

Console

- While configuring the job in the AWS Glue Console:
- Go to **Job Details** → **Job bookmark** → Choose:
 - Enable
 - Disable
 - Pause

Bookmark Modes

| Mode | Description |
|----------------|--|
| Enable | Track state across runs (recommended for incremental) |
| Disable | Ignores bookmark state — processes everything |
| Pause | Retains bookmark data, but ignores it during the current run |

You can switch back to "Enable" from "Pause" without losing bookmark history.

Bookmark in Action (Example)

```
dynamic_frame = glueContext.create_dynamic_frame.from_catalog(
    database="my_db",
    table_name="my_table",
    transformation_ctx="datasource0",
    additional_options={"jobBookmarkKeys": ["partition_key"], "jobBookmarkKeysSort0"
})
```

 This configuration:

- Processes only **new partitions** (partition_key)
- Sorts them in ascending order

Bookmark in S3 Context

When reading from S3, bookmarks track:

| Property | Purpose |
|-------------------------|-------------------------------|
| File path | Avoids re-reading same files |
| Last modified timestamp | Used for time-based filters |
| Custom prefix / suffix | Can influence inclusion logic |

 Example with S3 source:

```
dynamic_frame = glueContext.create_dynamic_frame.from_options(  
    connection_type="s3",  
    connection_options={  
        "paths": ["s3://my-bucket/input/"],  
        "recurse": True,  
        "groupFiles": "inPartition",  
        "groupSize": "1048576",  
        "enableUpdateCatalog": True  
    },  
    format="json"  
)
```

Bookmark Keys (For Catalog Sources)

| Key | Description |
|--------------------------|--|
| jobBookmarkKeys | Keys (columns/partitions) to track for bookmark purposes |
| jobBookmarkKeysSortOrder | "asc" or "desc" |

Used when pulling from Glue Catalog tables to identify what's new.

Bookmark Reset / Cleanup

To **reset bookmarks** and reprocess all data:

AWS Console:

- Go to **Glue Console** → **Jobs** → **Select Job** → **Job Runs**
- Click on **"Reset bookmark"**

Boto3:

```
import boto3
glue = boto3.client('glue')
glue.reset_job_bookmark(JobName='my-job-name')
```

Use Cases

✅ Recommended in:

- Daily ingestion from S3 buckets
- Batch updates from RDS/PostgreSQL
- Kafka or Kinesis streaming ETL (as checkpoints)

❌ Not ideal for:

- Full historical reprocessing
- Random, unstructured data flows without consistent timestamps/partitions

Behind the Scenes

Glue stores bookmark metadata in a hidden **DynamoDB-like system**. You can't access this directly, but it's visible via:

```
aws glue get-job-bookmark --job-name my-job-name
```

Response example:

```
{
  "JobBookmarkEntry": {
    "Version": 1,
    "Run": 15,
    "Attempt": 1,
    "JobName": "my-job-name",
    "JobBookmark": "<metadata_blob>"
  }
}
```

✅ Best Practices

- Always **test bookmark behavior** in Dev before enabling in Production
- Use **"Pause" mode** if you want to skip one run without losing state

- Apply **custom jobBookmarkKeys** when reading from partitioned tables
- Clean up unneeded bookmarks regularly

Would you like a **Python Glue job script** demonstrating incremental S3 or Redshift ETL with bookmarks and quality checks?